# 705. Design HashSet

| PLATFORM | DIFFICULTY | DATE |
| --- | --- | --- |
| Platform: LeetCode | EASY | Solved: July 3, 2026 |

**Problem Summary**

Design a HashSet without using built-in hash table libraries. Implement add(key), remove(key), and contains(key).

**Output**

Since keys are constrained to 0 <= key <= 10^6, use a direct-address boolean array of size 1000001. Each index represents a key — true means present, false means absent. No hashing needed at all.

**Key Insights**

- Slow/fast pointer trick: when fast reaches end, slow is at the midpoint.
- For odd-length lists, slow lands on the exact middle — that node is skipped naturally since we only iterate while right != null.
- Reversing second half reuses the **same three-pointer pattern** from LC 206.
- Comparison stops when right (shorter half) runs out — handles both odd and even correctly.

**Observations**

- No collision handling required — this is a degenerate case of hashing where hash(key) = key itself.
- Memory usage is fixed at ~1MB (1M booleans ≈ 1MB) — acceptable for this constraint.
- A real HashSet would use an array of buckets + chaining/open addressing for arbitrary keys.
- This approach would break immediately if key range were large (e.g., Long keys).

**Points to Remember**

1. **Direct addressing trick**: when key range is small and bounded, skip hashing entirely — use key as index directly.
2. Java boolean[] defaults to false — useful for presence/absence tracking without explicit initialization.
3. For a real HashSet implementation (interview follow-up), think: array of LinkedLists + key % capacity for bucket index + chaining for collisions.
4. This same pattern (boolean array indexed by value) appears in problems like "find duplicates", "count distinct elements", or simple frequency checks within a known range.

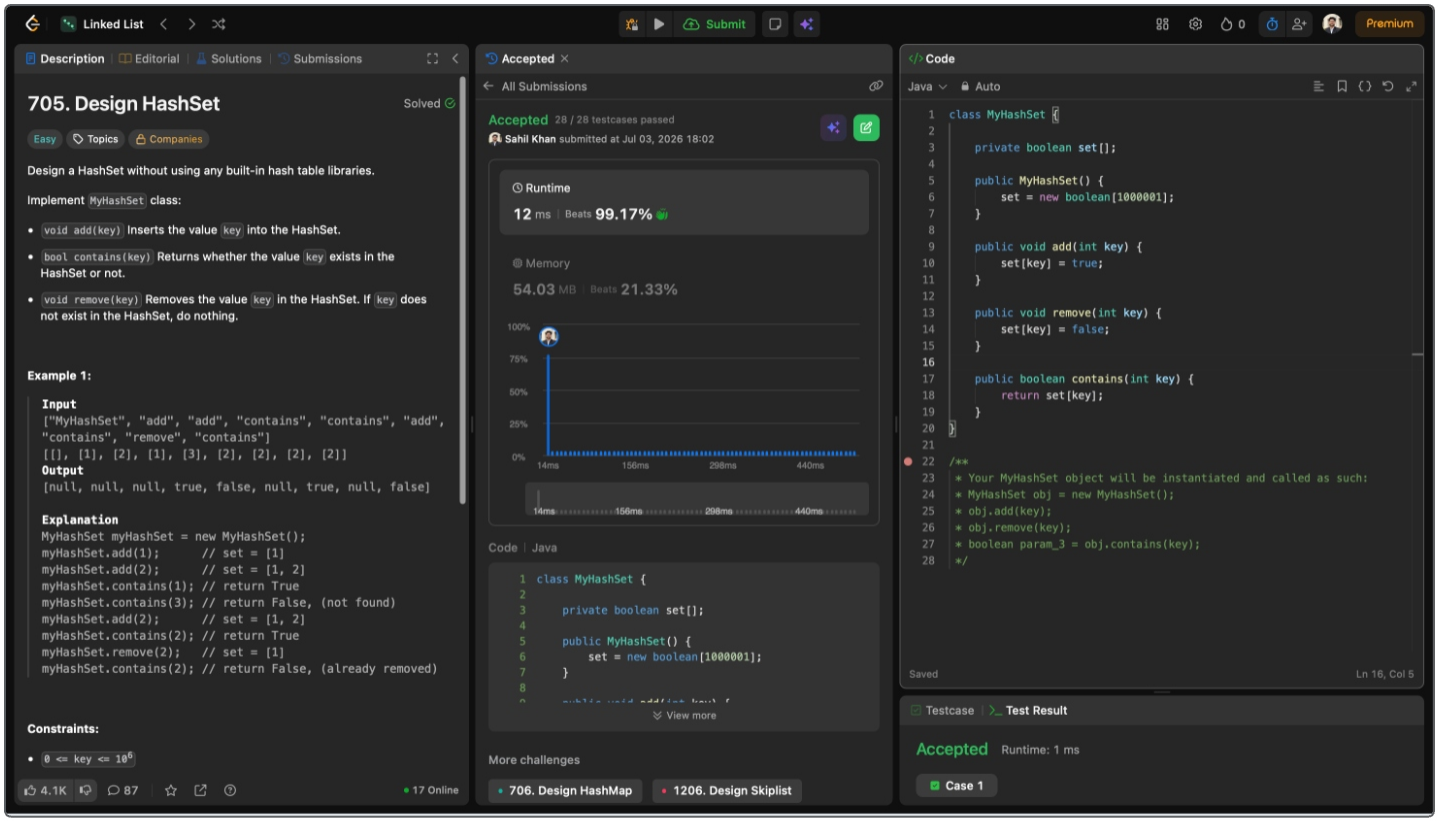
**Algorithm**

1. Initialize boolean[] set of size 1000001 (all default false).
2. add(key) → set set[key] = true.
3. remove(key) → set set[key] = false.
4. contains(key) → return set[key].

**Time Complexity & Space Complexity**

O(1) — all three operations are direct array access.

O(1) — fixed-size array of 1000001, independent of number of operations.


Linked List
Submit
Premium
Description | Editorial | Solutions | Submissions
705. Design HashSet
Solved
Easy
Topics
Companies
Design a HashSet without using any built-in hash table libraries.
Implement MyHashSet class:
void add(key) Inserts the value key into the HashSet.
bool contains(key) Returns whether the value key exists in the HashSet or not.
void remove(key) Removes the value key in the HashSet. If key does not exist in the HashSet, do nothing.
Example 1:
Input
["MyHashSet", "add", "add", "contains", "contains", "add", "contains", "remove", "contains"]
[[], [1], [2], [1], [3], [2], [2], [2], [2]]
Output
[null, null, null, true, false, null, true, null, false]
Explanation
MyHashSet myHashSet = new MyHashSet();
myHashSet.add(1); // set = [1]
myHashSet.add(2); // set = [1, 2]
myHashSet.contains(1); // return True
myHashSet.contains(3); // return False, (not found)
myHashSet.add(2); // set = [1, 2]
myHashSet.contains(2); // return True
myHashSet.remove(2); // set = [1]
myHashSet.contains(2); // return False, (already removed)
Constraints:
0 <= key <= 10^6
4.1K
87
17 Online
Accepted
All Submissions
Accepted 28 / 28 testcases passed
Sahil Khan submitted at Jul 03, 2026 18:02
Runtime
12 ms | Beats 99.17%
Memory
54.03 MB | Beats 21.33%
100%
75%
50%
25%
0%
14ms
156ms
298ms
440ms
Code | Java
class MyHashSet {
private boolean set[];
public MyHashSet() {
set = new boolean[1000001];
}
View more
More challenges
706. Design HashMap
1206. Design Skiplist
Code
Java
Auto
class MyHashSet {
private boolean set[];
public MyHashSet() {
set = new boolean[1000001];
}
public void add(int key) {
set[key] = true;
}
public void remove(int key) {
set[key] = false;
}
public boolean contains(int key) {
return set[key];
}
}
/**
* Your MyHashSet object will be instantiated and called as such:
* MyHashSet obj = new MyHashSet();
* obj.add(key);
* obj.remove(key);
* boolean param_3 = obj.contains(key);
*/
Saved
Ln 16, Col 5
Testcase | Test Result
Accepted Runtime: 1 ms
Case 1